



Artificial Intelligence Foundations (CIS4049-N)

Lab Introduction

Module Leader: Dr Alessandro Di Stefano

Email: A.DiStefano@tees.ac.uk

School of **Computing, Engineering & Digital Technologies**

tees.ac.uk/computing

Lab Session - Introduction

- **Uninformed Searching Algorithms - Implementation with both R and Python**
 - **Depth-First Search (DFS)**
 - **Breadth-First Search (BFS)**
 - **Depth Limited Search (DLS)**
 - **Uniform Cost Search (UCS)**
 - **Iterative Deepening Depth First Search (IDDFS)**
 - **Bidirectional Search (BS)**
 - **Dijkstra algorithm (variant of UCS)**

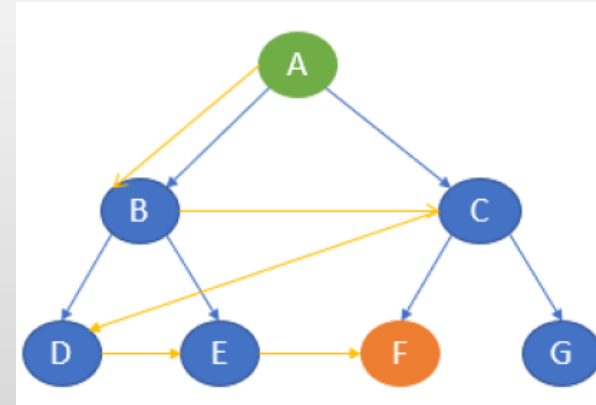
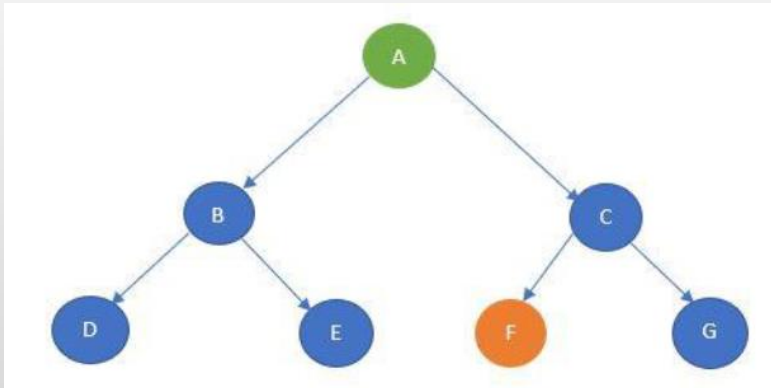
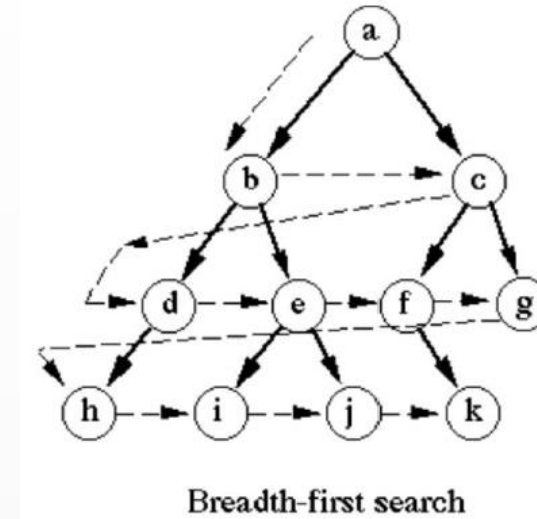
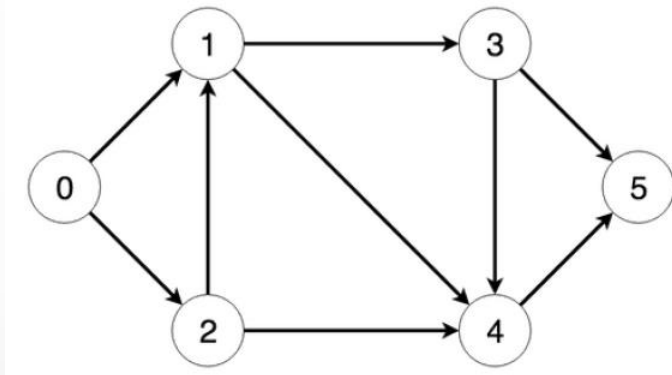
Uninformed Searching Algorithms

- **Description** of the algorithm
- **Example**
- **Considerations** about the **runtime complexity** and **space complexity**
- **Implementation in both Python and R** – pseudocode and concrete implementation in both languages
- **Advantages/drawbacks** and **final considerations**

Breadth-First Search (BFS)

- **Description** of the algorithm

- **Example**



Breadth-First Search (BFS)

- Pseudocode of the algorithm

```
def bfs(graph, start):  
    """  
    Implementation of Breadth-First-Search (BFS) using adjacency matrix.  
    This returns nothing (yet), it is meant to be a template for whatever you want to  
    do with it,  
    e.g. finding the shortest path in a unweighted graph.  
    This has a runtime of  $O(|V|^2)$  ( $|V|$  = number of Nodes), for a faster implementation  
    see @see ../fast/BFS.java (using adjacency Lists)  
    :param graph: an adjacency-matrix-representation of the graph where (x,y) is True  
    if the the there is an edge between nodes x and y.  
    :param start: the node to start from.  
    :return: Array array containing the shortest distances from the given start node to  
    each other node  
    """  
    # A Queue to manage the nodes that have yet to be visited, intialized with the  
    start node  
    queue = [start]  
    # A boolean array indicating whether we have already visited a node  
    visited = [False] * len(graph)  
    # (The start node is already visited)  
    visited[start] = True  
    # Keeping the distances (might not be necessary depending on your use case)  
    distances = [float("inf")] * len(  
        graph) # Technically no need to set initial values since every node is visted  
    exactly once  
    # (the distance to the start node is 0)
```

```
distances[start] = 0  
# While there are nodes left to visit...  
while len(queue) > 0:  
    print("Visited nodes: " + str(visited))  
    print("Distances: " + str(distances))  
    node = queue.pop(0)  
    print("Removing node " + str(node) + " from the queue...")  
    # ...for all neighboring nodes that haven't been visited yet...  
    for i in range(len(graph[node])):  
        if graph[node][i] and not visited[i]:  
            # Do whatever you want to do with the node here.  
            # Visit it, set the distance and add it to the queue  
            visited[i] = True  
            distances[i] = distances[node] + 1  
            queue.append(i)  
            print("Visiting node " + str(i) + ", setting its distance to " + str(  
                distances[i]) + " and adding it to the queue")  
  
    print("No more nodes in the queue. Distances: " + str(distances))  
    return distances
```

Breadth-First Search (BFS)

- Implementation of the algorithm

```
# sample graph implemented as a dictionary - The illustrated graph is represented using
an adjacency list. An easy way to do this in Python is to use a dictionary data
structure, where each vertex has a stored list of its adjacent nodes.
```

```
graph = {
    'A' : ['B','C'],
    'B' : ['D', 'E'],
    'C' : ['F'],
    'D' : [],
    'E' : ['F'],
    'F' : []
}
```

```
visited = [] # visited is a list to keep track of visited nodes.
```

```
queue = [] #Initialize a queue - queue is a list that is used to keep track of nodes
currently in the queue
```

```
# The arguments of the following bfs function are the visited list, the graph in the
form of a dictionary, and the starting node A. bfs follows the algorithm described
above - 1. It checks and appends the starting node to the visited list and the queue.
2. Then, while the queue contains elements, it keeps taking out nodes from the queue,
appends the neighbours of that node to the queue if they are unvisited, and marks them
as visited. 3. This continues until the queue is empty.
```

```
def bfs(visited, graph, node):
    visited.append(node)
    queue.append(node)

    while queue:
        s = queue.pop(0)
        print (s, end = " ")

        for neighbour in graph[s]:
            if neighbour not in visited:
                visited.append(neighbour)
                queue.append(neighbour)
```

```
# Driver Code
```

```
bfs(visited, graph, 'A')
```

Comparison of uniformed search algorithms

- Final interpretations

Criterion	Breadth-First	Uniform-Cost	Depth-First	Depth-Limited	Iterative Deepening	Bidirectional (if applicable)
Complete	Yes ^a	Yes ^{a,b}	No	No	Yes ^a	Yes ^{a,d}
Time Complexity	$O(b^d)$	$O(b^{(c/E)})$	$O(b^m)$	$O(b^l)$	$O(b^d)$	$O(b^{d/2})$
Space Complexity	$O(b^d)$	$O(b^{(c/E)})$	$O(bm)$	$O(bl)$	$O(bd)$	$O(b^{d/2})$
Optimal	Yes ^c	Yes	No	No	Yes ^c	Yes ^{c,d}

b -> branching factor
 d -> depth of the shallowest solution
 m -> max. depth of search tree
 l -> depth limit

a -> complete if b is finite
 b -> complete if step costs $\geq E$ for $E > 0$
 c -> optimal if step costs are same
 d -> if both directions use BFS

Wrap-up

- **Description and implementation** of Uninformed Search algorithms both in R and Python
- **Advantages/drawbacks and final considerations (runtime and space complexity)**
- **Comparison of uninformed search strategies** - Final interpretations

Questions?

A.DiStefano@tees.ac.uk